SE/COM S 3190 - Construction of User Interfaces

Final Project Technical Report
Spring 2026

WatchVault

Team Members

Member 1 Name
munozo@iastate.edu

Alex Erickson
aerick22@iastate.edu

Iowa State University
Department of Computer Science

Contents

1 Introduction

Use this section to set the foundation for your project. Provide a clear explanation of what your application is, why it exists, and who it is meant to serve.

Watchvault is an app where users can view trending films, save their favorite movies, and create and view reviews by other users. Any user can create their own set of watched movies where they must set the individual status to watched to create a review. Admins can manually delete reviews they deem inappropriate and delete any film in the local database as well as request one through TMDB's api. Every film on the app has its own individual review page which compiles and filters reviews through multiple pages; it incorporates a social feature through a like and dislike system.

The purpose of watchvault is to provide an intuitive and quick way to save films, track sequels and spin-offs, and write reviews for others to interact with.

1.1 Project Overview

Explain the overall purpose of your application. Include instructions such as:

- The problem statement
  Users often have no centralized way to track movies they want to watch, movies they have watched, and the opinions or reviews associated with them. Movie discovery, personal tracking, and community reviews are typically fragmented across many platforms.
- Describe why this problem is relevant or important
  The rapid growth of streaming platforms and constantly expanding film libraries, users need an organized way to manage what they watch. Without a unified system, it becomes difficult to track sequels, revisit films, or engage meaningfully with other users opinions.
- Summarize what your application does at a high level (1–2 sentences)
  Watchvault is a movie and review web application where users can browse trending films, maintain personalized watchlists, and write or view community reviews for each movie. It includes a minor social interaction feature through likes and dislikes on reviews.
- Mention the scope of your solution (e.g., web application, interactive tool, system dashboard)
  Watchvault is a full-stack web application that integrates external movie data from TMDB with a custom backend system for users, watchlists, movies, and reviews. It

supports user authentication, CRUD operations for watchlists and reviews, and administrative moderation features for managing reviews and movies.

1.2 Target Users and Use Cases

Describe the intended audience for your system. For example:

- Identify who will benefit from using your application

  Movie fanatics that want to quickly browse through an endless selection of films while also receiving catered recommendations automatically.
- Explain typical scenarios or workflows where the system is used

  A user logs in to mark off a movie on their watchlist, they don't feel like making a review today but they check the recommendations page and notice "whoah this movie has a sequel and a spin off". Okay they feel like documenting a chronological journey of a series watch through. They write a review on the movie they watched and see someone has a similar experience, they like their review and right before they log off they return to the recommendations page because they forgot to add the movie to their watchlist.
- Include examples of both primary users (main audience) and secondary users (admins, support staff, etc.)

  A user writes an inappropriate review on the movie "Dune 3" an admin steps in and manually reviews and deletes their review. The admin realizes hmm we don't have the original Dune movies in the database, he goes to the search page and manually adds it through the TMDB api, the day is saved.

1.3 Core Features

List the major capabilities of your system. Suggested points to address:

- Describe each key feature concisely (e.g., CRUD operations, dashboards, search mechanisms)

  4 Core features (not incld. Login, Sign Up, Admin)

  - **Search page:** Movies are retrieved(get), can be added by admins, can be deleted by

admins, and updated (synced) with our backend local instance.

- Watchlists: Watch lists can be retrieved, remove films, add films, and update their status to “watched”

- Recommended: Recommendations are retrieved by accessing a user's genre preferences and current watchlist. They are updated when users add or remove movies. Recommendations are purged (deleted) if a user has no movies, minus the top rated and random recs page.

- Reviews: Reviews can be created if a user has set a movie to “watched”, reviews can be edited by their respective creator, reviews can be manually deleted by their creator or an admin. Reviews are fetched for individual movies by clicking on their movie card.

Mention any unique or advanced functionality your team implemented

Admins can request a movie on the NO RESULTS page of Search through the TMDB api and it is correctly added to the localdb. Using a secret password a user can sign in, promote themselves to gain specific privileges like movie deletion, addition, and review removal. Recommended movies pull from the TMDB api and if a user adds them to their watchlist they are also added to the localdb.

- Clarify how these features solve the problem identified

1.4 Originality vs Inspiration

Explain your project’s origin. Instructions to include:

- State whether the idea is entirely original or inspired by an existing application

  The app is loosely based on the Internet Movie DataBase, in fact we pull their ratings and general movie data through TMDB a free limited api based on IMDB.

- If inspired, identify what aspects influenced your design

  The review creation and review pages are based on IMDB (loosely) the recommendation page idea is as well but the layout shares no similarities.

- Describe how your version differs or improves upon existing alternatives
  Our app is much more simplified to the core aspects of say IMDB or letterboxd so to remain quick; it lazily caches new movies not in the localdb. It strips a lot of the fat that IMDB has and focuses on the main aspects of film obsession/discussion, minimal user interaction, and a real feel to the review pages.

2 System Overview and Project Description

Use this section to give a structured explanation of how your system is built and how it behaves from both a technical and user perspective.

2.1 Major Functional Modules

Break your system into logical modules. Provide instructions such as:

- Identify the core modules or components of your app (e.g., Authentication, Search, Dashboard, Admin Tools): Login, Signup, Recommendations, Reviews, Watchlist, Home, Search, Setttings and User
- Briefly describe what each module is responsible for: **Login is for the user to login if they have an account, Signup if the user who don't have an account, they can register, Recommendations will give the user movies based on favorite genres, and what's in their watchlist, Reviews allows users to see reviews of a certain movie from other users on the site. Home, the landing page, tells the user what we are about and shows some random movies from our database. Search allows a user to search for a movie in the database. Settings allow a user to edit their information as well as set favorite genres and delete or log out of their account. And User which is a Provider that allows us to use its context throughout the whole website**
- Explain how modules communicate with each other if applicable: **They communicate through the home screen, which includes buttons and a search bar that lead to components, and a navbar, which is in Layout.jsx.**
- Mention any key design decisions behind this modular structure: **User.jsx was created because we would have had to use many redundant props for a user. This makes a very concise and easy way to get the user who is logging in.**

2.2 UI and Navigation Flow

Describe how users move through your application. Suggested instructions:

- Outline the path a typical user follows when using your system: **User goes to login which can be access anywhere because all components require the user to be logged in to access. If the user is new, they can click a nav link at the bottom of the login page to sign up. Depending on that, they will either log in or sign up. Signing up will take them to the login page, where they will have to log in again. They can either search or browse recommendations based on their watchlist or favorite genres, which they can enter on the settings page. They can then add a movie to the watchlist. After adding, they can go to the watchlist page to see it. If they have watched it, they can click watch on the movie. After watching the movie, they can ge write a review and then write a review of whatever movie they have watched. After leaving a review, they can find that review by clicking the movie itself.**
- Explain the purpose of each main page or UI view: **Login is for the user to login if they have an account, Signup if the user who don't have an account, they can register, Recommendations will give the user movies based on favorite genres, and what's in their watchlist, Reviews allows users to see reviews of a certain movie from other users on the site. Home, the landing page, tells the user what we are about and shows some random movies from our database. Search allows a user to search for a movie in the database. Settings allow a user to edit their information as well as set favorite genres and delete or log out of their account. And User which is a Provider that allows us to use its context throughout the whole website**
- Include conditional flows (e.g., what happens after login, or when data is submitted): **Each component, if the user is not logged in, will show them a card that says they need to log in and direct them to the login via nav link. Also, all admin features are redner condition; if the role is admin, then the user can see the delete movie and the admin control panel**
- If helpful, reference diagrams or screenshots later in the document

2.3 CRUD Entities

Document the entities your system manages. Provide guidance such as:

- List each entity that supports Create, Read, Update, Delete operations: Movies,Users, Reviews, Keys, and Watchlist

1. Movies

Data Fields:

- **id** — TMDB movie ID (numeric, unique)
- **title**, description, director
- **rating**, voteCount
- **genres** — array of genre name strings
- **poster**, **backdropPath** — image URLs
- **releaseYear**, **releaseDate**
- **runtime, status**
- **createdAt, updatedAt**

2. Users

Data Fields:

- **userId** — auto-incrementing integer
- **username**, **email**, **password** (bcrypt hashed)
- **watchlistId** — FK to the user's Watchlist
- **favGeneres** — array of preferred genres
- **role** — "user" or "admin"
- **createdAt**

Actions / CRUD:

- **Create** (POST /auth/register) — registers a new user and simultaneously creates a linked Watchlist

- **Read** (POST /auth/login) — authenticates and returns a safe user object (no password)
- **Read** (GET /auth/admin/users) — admin view of all users
- **Update** (PUT /auth/user/:id) — lets a user edit their profile (username, favGeneres, etc.);
- **Update** (POST /auth/admin/users/:id/role) — admin-only role promotion/demotion
- **Delete** (DELETE /auth/user/:id) — removes the user and cascades to delete their Watchlist

Actions / CRUD:

- **Read** (GET /search/:id, GET /search) — look up a single movie or search by title with pagination; powers the browsing/search UI
- **Read** (GET /movies/tmdb-search) — live TMDB search for movies not yet in the local DB; used when a user wants to request a missing title
- **Create** (POST /movies/request) — pulls full movie details from TMDB and inserts it into the local DB; allows the catalog to grow on demand
- **Delete** (DELETE /movies/remove) — removes a movie from the local vault; admin-level cleanup
- **Update** (PUT /movies/update) — bulk-syncs the DB against TMDB's top_rated/popular/upcoming endpoints and patches stale records; keeps ratings, metadata, and posters fres

- If relevant, mention relationships between entities

3 File and Folder Architecture

This section documents the physical layout of your project. Explain the reasoning behind your folder choices and how the structure supports clean development practices.

3.1 Folder Structure Overview

Provide a high-level explanation of how your project is organized. You may include points such as:

- Why the project uses separate backend/ and frontend/ directories

  To distinguish between functionality and to assist in developing large projects we use independent backend and frontend folders. Subfolders exist to show a topdown development of the project, with our index and home features being higher.
- How this structure improves maintainability or workflow

  By ensuring files can be sensibly and quickly found while not being obscured in multiple folders we only nest on average 3-4 times and with minimal instances.
- Any alternative structures you considered and why this one was chosen

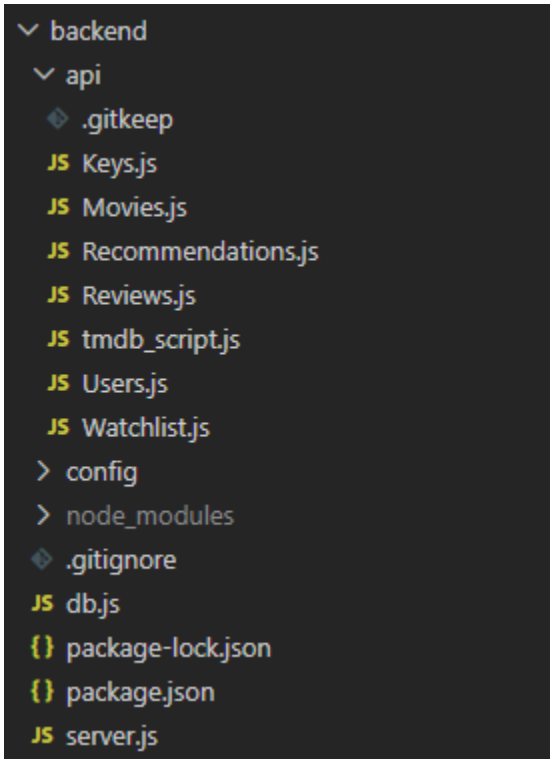
  We did not consider any drastic alternatives, we may have made some adjustments on the way like storing our movies(1000 objects) at the highest level. Another example is the four similar recommendations in a subfolder due to the fact that they share the same page but require different formatting.

3.2 Backend Folder Structure

3.2.1 Backend Folder Tree

Insert a picture of your backend folder structure here. Include all files and subdirectories

relevant to your server, routes, controllers, configuration, and environment.



3.2.2 Backend Folder Description

Write a short explanation for each backend folder or file. Suggested topics to cover:

- What each folder contains and why it exists

  The api folder contains a separate CRUD for each of the tables in our database to help in development and responsive UI features alongside backend integration.
- How controllers, routes, and configuration files are separated

  Controllers are identified by the capital letter in the api letter corresponding to their table. Routes are found in the server folder and our database is initialized in the db folder.
- Why this structure supports good API design

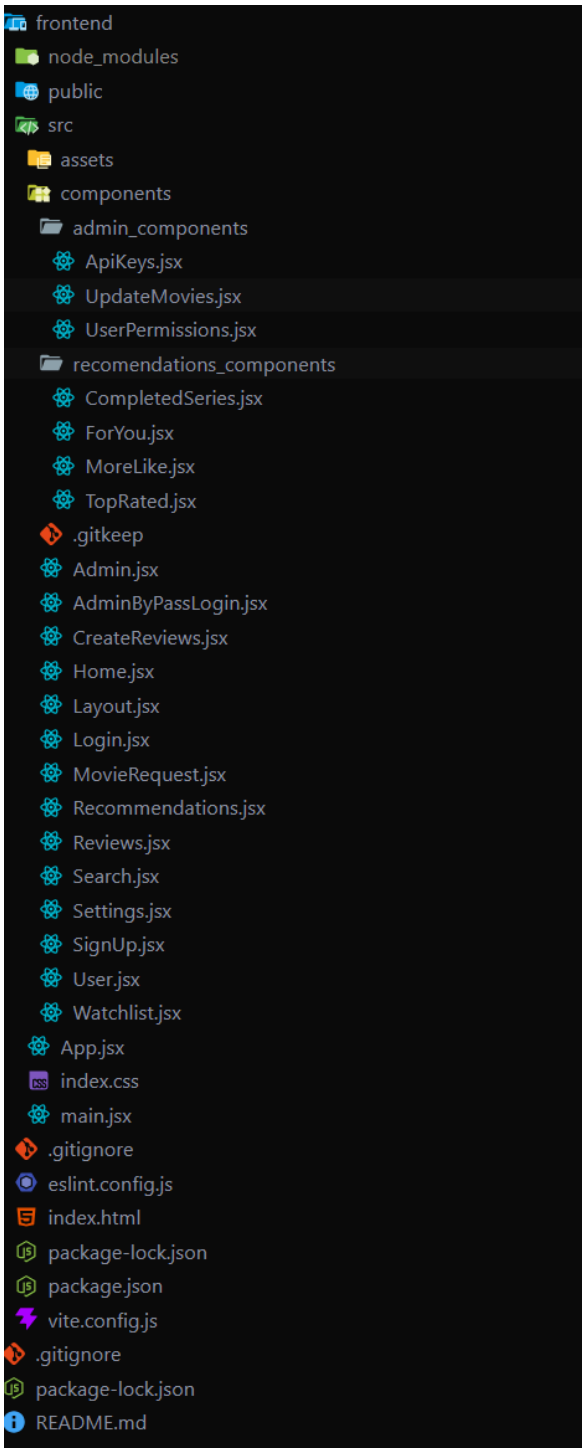
  It's separated cleanly, api (controllers), server, and database 3 clear structures working together in the backend.
- Any naming conventions or organizational standards your team followed

  Capitalization of the controllers which are equivalent in naming to their database table.

3.3 Frontend Folder Structure

3.3.1 Frontend Folder Tree

Insert a picture representing your actual frontend directory. Mirror the style used in thebackend diagram but modify according to your fronte



3.3.2 Frontend Folder Description

Describe each major folder or file in the frontend. Suggested points:

- Purpose of each folder (components, pages, hooks, context, utils, etc.): components store the main views of the website, such as Home, Login, and Recommendations. Admin components and recommendations components store subviews on the admin and recommendations pages.
- Why did the team organize the frontend this way: Easier navigation and readability
- Any patterns used (e.g., component-based design, custom hooks) component-based design/view-based
- How does this structure help scalability or readability? It makes the files less cluttered so that a developer, code review, or us as the creators easier.

4 Code Explanation and Logic Flow

This section explains how the application behaves internally, how data flows between components, and how requests are processed from the UI to the database.

4.1 Frontend–Backend Interaction

Describe the request/response cycle in detail, including:

- How the frontend triggers API calls (e.g., using fetch)
  We'll use SEARCH for example, our search page attempts to populate the frontend by initially calling useEffect which calls a function to fetchMovies. Then queries 35 movies(7x5 row,column) by using a route established in the backend for individual movies, Search then returns a bunch of card instances of movies organized in a neat fashion using our index.css file as well as Layout, Container, and Card hierarchies .
- How data is sent to the backend (JSON bodies, query parameters, route parameters)
  I'll use SEARCH's handleDeleteMovie function. Movie cards on the search page conditionally render a red 'X' button which admins can see and interact with. When the button is pressed a confirmation is rendered and the delete function call our endpoint '/api/movies/remove/' using the id stored in the card data to delete

that specific movie from the localdb instance.

- How responses are handled in the React components (state updates, conditional rendering)
  If a user attempts to add a movie already in their watchlist you will be told accordingly, major actions have confirmation windows. Like/dislike can only be interacted with by non review creators, users can delete their own reviews, admins can delete any review. Users can edit their own reviews, and a special NO RESULTS page renders for admins to add movies using TMDB's api.

- Error-handling strategies (try/catch blocks, UI messages, fallback states)
  If the backend server is not running you will find out very quickly through error handling on missing endpoints. Incorrect passwords and usernames are handled accordingly and missing review titles and descriptions provide UI warnings. The recommendations page renders disclaimers if you don't have anything in your watchlist to recommend.

4.2 React Component Example

Insert a code snippet of a meaningful React component and explain:

```
export default function CreateReviews() {
    const { user } = useContext(UserContext);
    const [watchlist, setWatchlist]  = useState([]);
    const [itemsChecked, setItemsChecked] = useState([]);
    const [loading, setLoading] = useState(false);
    const [submitting, setSubmitting] = useState(false);
    const [error, setError] = useState(null);
    const [success, setSuccess] = useState(null);
    const navigate = useNavigate();

    const [selectedMovie, setSelectedMovie] = useState('');
    const [title, setTitle] = useState('');
    const [description, setDescription] = useState('');
    const [rating, setRating] = useState('');
    const [recommend, setRecommend] = useState(false);

    useEffect(() => {
        if (!user) { navigate('/login'); return; }
        fetchWatchlist();
    }, [user]);

    const fetchWatchlist = async () => {···
    };

    // Only movies marked as watched are reviewable
    const watchedMovies = watchlist.filter((movie) =>
        itemsChecked.includes(movie.id)
    );

    const handleSubmit = async (e) => {
        e.preventDefault();
        if (!selectedMovie) return setError("Please select a movie to review.");
        if (!title.trim()) return setError("Please enter a review title.");
        if (!description.trim()) return setError("Please enter a description.");
        if (!rating) return setError("Please select a rating.");

        const movie = watchedMovies.find((m) => m.id === Number(selectedMovie));
        if (!movie) return setError("Selected movie not found.");

        setSubmitting(true);
        setError(null);
        setSuccess(null);
```

- The purpose of the component within the UI

  This is for the review creation feature, it handles input and drop down window selection for ratings and movie selection.

- Props received and the flow of data downward

- Hooks used (useState, useEffect, custom hooks)

  useContext, useState, useEffect

- Form handling logic, validation, and state updates

  Form handling is pretty on rails we do a simple input check ensuring users are not submitting reviews with unfinished fields. Unfinished submissions are not allowed, there's no advance input checking but the user receives an alert when their valid review is processed. Reviews are then reset using useState() by setting them as empty.

  UseEffect is used to fetch the login status of a user, if they are not logged in they cannot access the page.
- How it interacts with backend routes (e.g., making POST/PUT requests)

  On review submission a POST request is sent to the backend (/api/reviews) as described in the previous question fields are checked for empty.
- Conditional UI logic (loading states, error displays, success messages)

  A success message is conditionally rendered with useState, the appropriate movie title is also given "Your review for *movie* was submitted!".

4.3 API Route Examples

Provide one example for each CRUD-related route type (POST, GET, PUT, DELETE). For each route, describe:

Review Creation CRUD

GET /reviews/:movieId: Fetches all reviews for a given movie using movieId in the URL, performs no validation or middleware, queries the database for matching reviews sorted by newest first, returns 200 OK with an array of reviews (or empty array), and returns 500 on database failure.

POST /reviews: Creates a new review with required fields in the request body (movie and user info, title, description, rating), validates required fields, rating range (1-10), and duplicate user review, uses no middleware, runs a duplicate findOne then insertOne, returns 201 Created with the new review object, and returns 400, 409, or 500 for validation, duplicates, or DB errors.

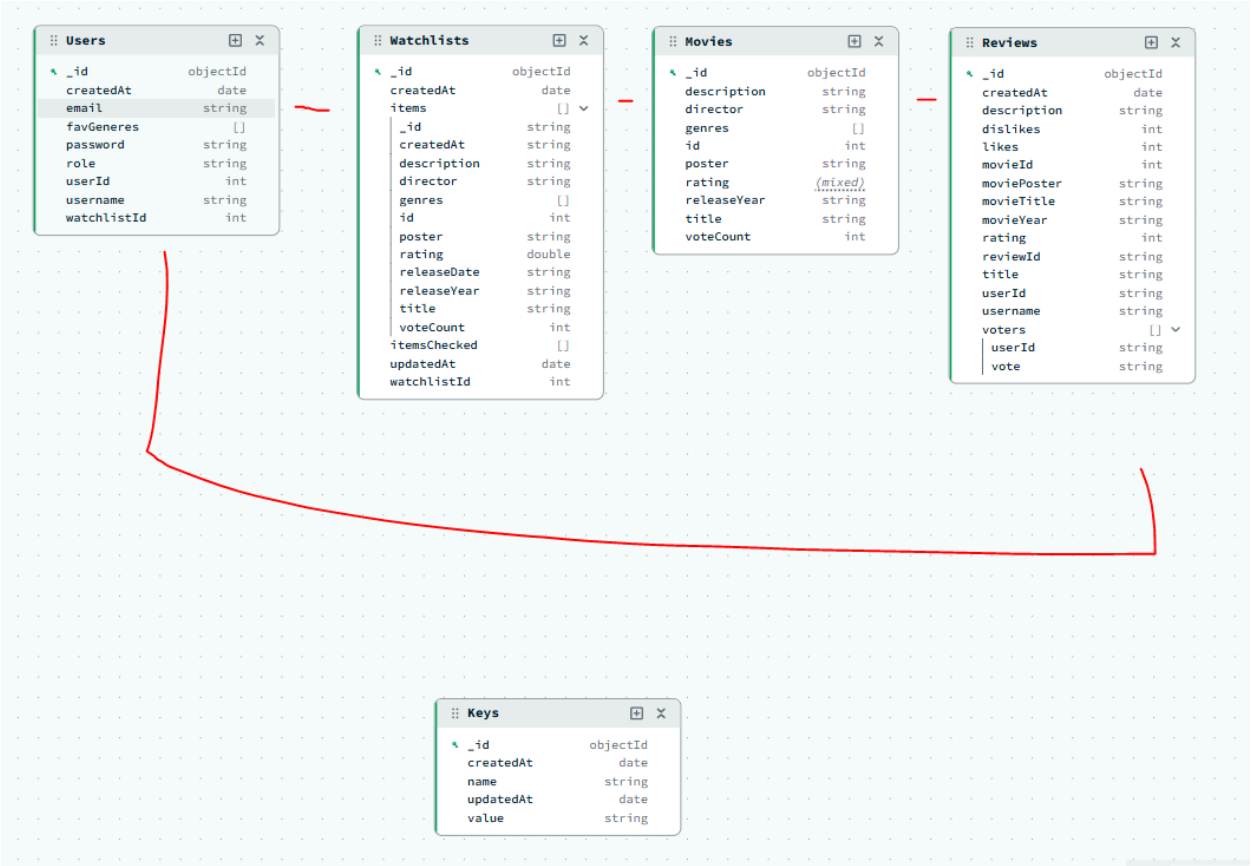
PUT /reviews/:reviewId: Updates an existing review by reviewId in the URL using optional fields in the body plus required userId, validates user presence, rating range, and ownership, uses no middleware, performs a findOne then findOneAndUpdate, returns 200 OK with the

updated review, and returns 400, 403, 404, or 500 depending on validation, authorization, or DB failure.
DELETE /reviews/:reviewId: Deletes a review by reviewId in the URL using userId in the body, validates user presence and ownership (or admin role), uses no middleware, performs a findOne then deleteOne, returns 200 OK with a success message, and returns 400, 403, 404, or 500 for missing data, unauthorized access, not found, or DB/runtime errors.

- The purpose of the endpoint
- Expected request format (body schema, params)
- Validation steps performed
- Any middleware used (auth middleware, validation middleware)
- Database operations triggered by the route
- Expected response structure and status codes
- How errors are handled and returned to the frontend

4.4 Database Interaction

Show your schema (Mongoose model or SQL table) and explain:



- The fields stored in the database and their types

  Users table is self explanatory with basic information about their favorite genres, and a link to their respective watchlist. Watchlists contain movie data, the time they're created, and updated. Movies contain some stripped down information from TMDB, image link, director, title, rating, year etc. Reviews need to store movie objects since they display their information, when a review page is accessed it lazily stores the information.
- Relationships between collections/tables (if applicable)

  Users 1--1 Watchlists

  Watchlists M--N Movies

  Users 1--N Reviews

  Movies 1--N Reviews
- Validation rules or constraints at the database level

  Basic validation is handled in the backend routes.Required fields are checked before inserts or updates, invalid IDs are rejected, and duplicate watchlist entries are prevented
- Common queries used (find, update, aggregate, etc.)

  findOne(): Users, Keys etc.

  countDocuments(): Movies

  insertOne(): Movies, Reviews… etc.

  Aggregate(): Movies

  deleteOne(): Reviews and Movies

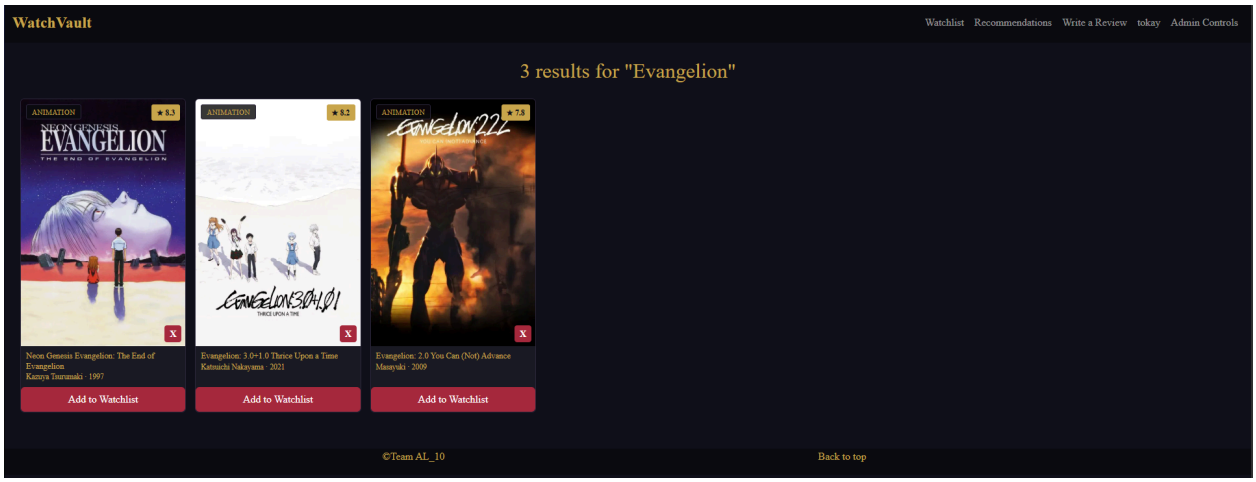
  findOneAndUpdate(): Reviews
- Any indexing or query optimization considerations

  No custom indexing or query optimizations were implemented, the only used is MongoDB's default_id indexing

5 Screenshots and Web Views

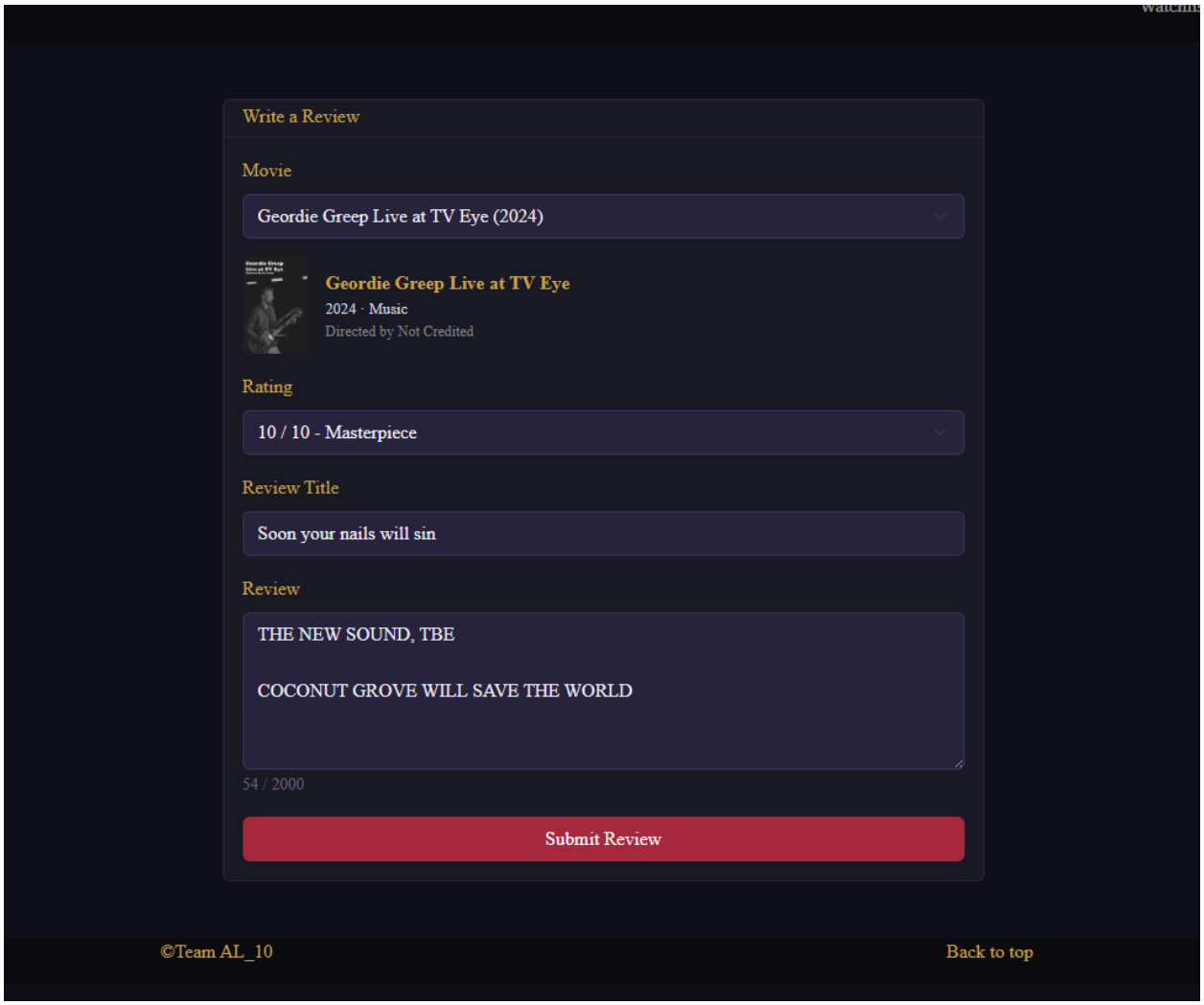
Include screenshots of each important page in the application. For each screenshot:

SEARCH



- The purpose of this page is to provide search feedback on any movie in the localdb instance, for example here's 3 movies all sharing the same title. The original and a reimagining alongside its sequels.
- The key UI elements shown are movie cards, movie deletion button, and add to watch list buttons.
- Users can add movies to their watchlist, admins can remove them, searches occur in the previous screen.
- After deletion, movies are removed from the database and the card stops rendering, after additional movies are added to your list and your are given a green alert message, GET, POST, PUT, DELETE are all possible on the search page.

CREATE REVIEW



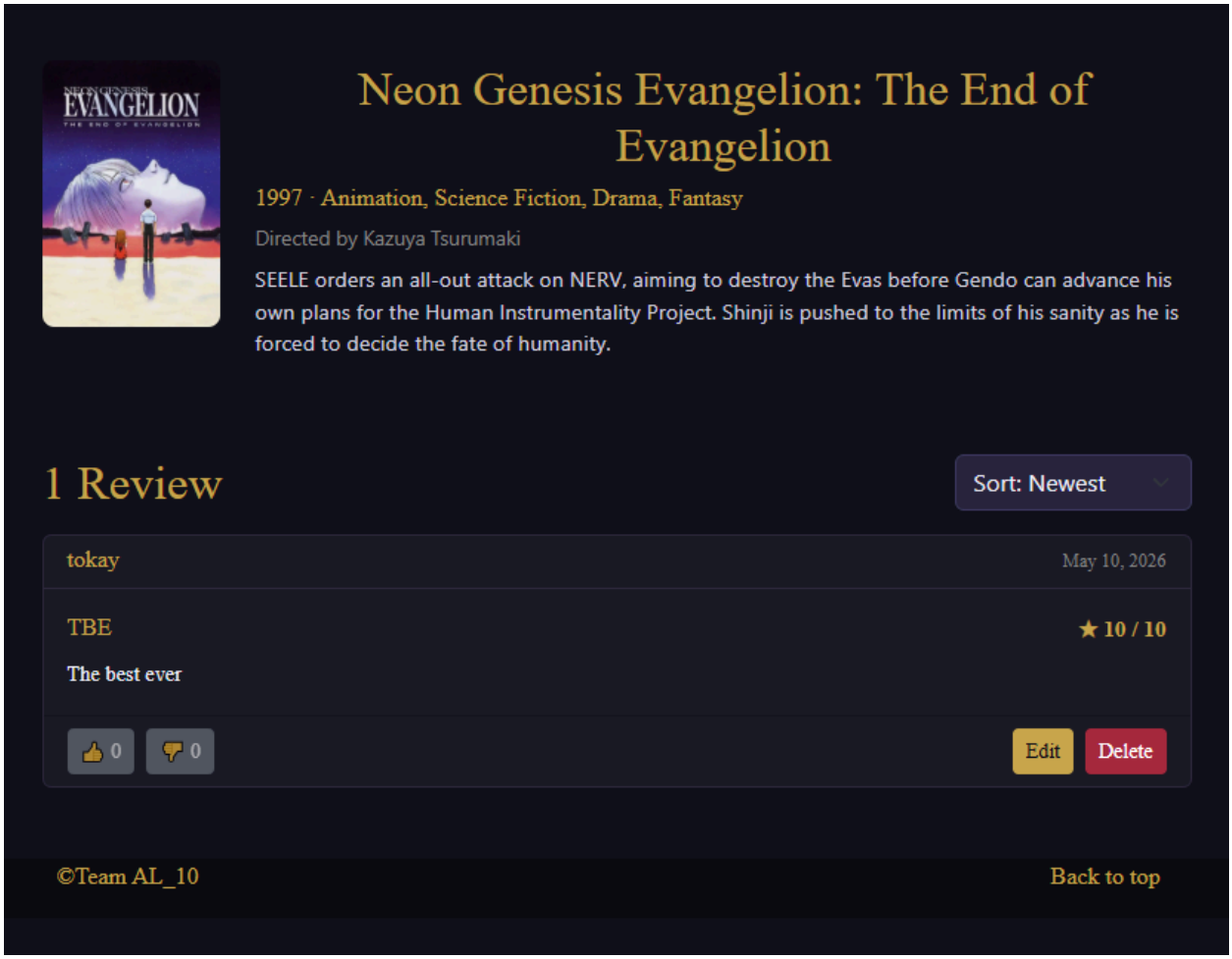
- The purpose of this page is to allow users to create reviews on movies they've marked off as watched in their watchlist.

- They key UI elements are mentioned below, but the most significant is the review submission which sends POST requests to the backend to create a review to the specific movie selected.

- Users can rate, title, review, and select a movie on this page.

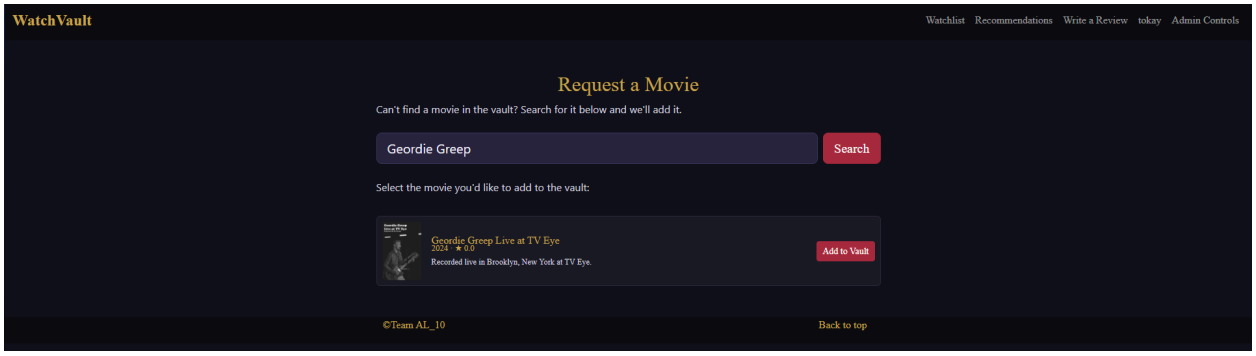
- Movie selection and Rating are dropdown menus which give you static choices, title and review are input boxes. On review submission the database is updated with a new review for that particular film.

REVIEW



- The purpose of this page is to compile and filter through reviews users submit, alongside basic movie information.
- The key UI elements are Like/Dislike, editing, deletion, sorting, and page navigation. Pages store 5 reviews and sorting is page wise.
- The user can edit all aspects of their review, they cannot like or dislike their own posts, and the admin can even delete any person's review.
- When a user likes a review the database sends a PUT request to update the review's vote counts. When a user edits their review a dropdown menu and two input boxes are rendered in place of the review itself.

MOVIE REQUEST



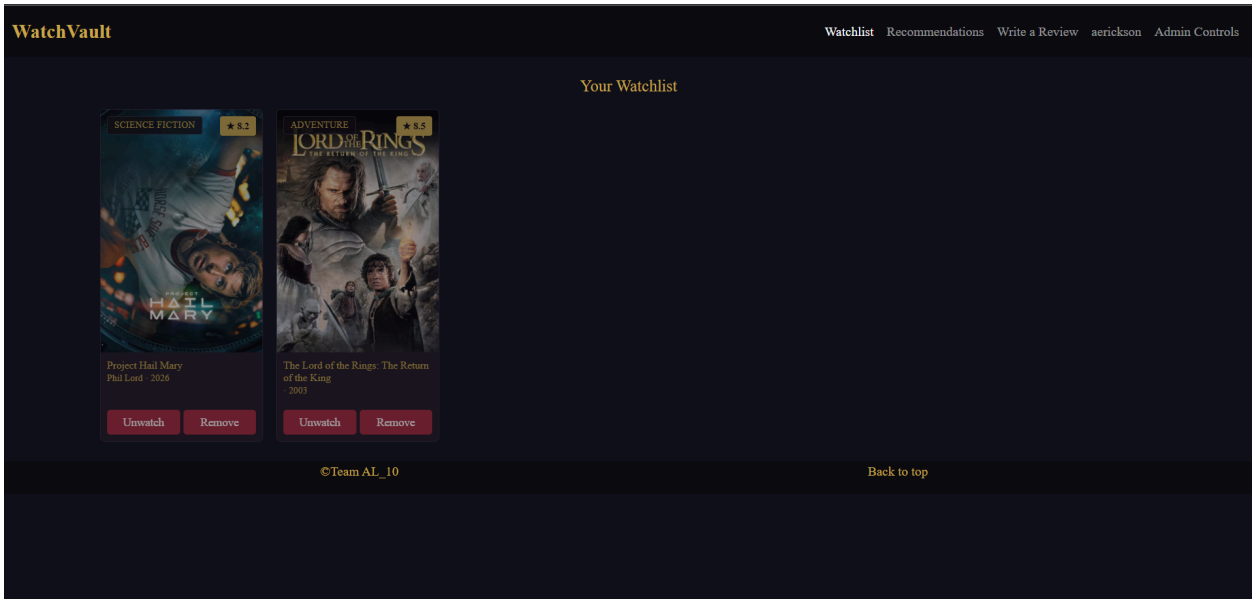
- The purpose of this page is to allow admins to submit a POST request to update the localdb instance using the TMDB api which contains around 1.3 million movies. IT CAN ONLY BE ACCESSED BY ADMINS ON THE "NO RESULTS" PAGE.

- They key UI elements are Searching which is limited at 5 seconds per search, and add button which adds the movie locally for search, review, and watchlist support.

- The user can add multiple movies related to their specific search at once and they can search for practically any movie in existence including niche videos like that of Geordie Greep's incredible live performance.

- After searching, the api key I generated is used to request 10 movies from TMDB most similar to your search. When adding a movie a POST request is sent to the server to add to your localdb instance.

WATCHLIST

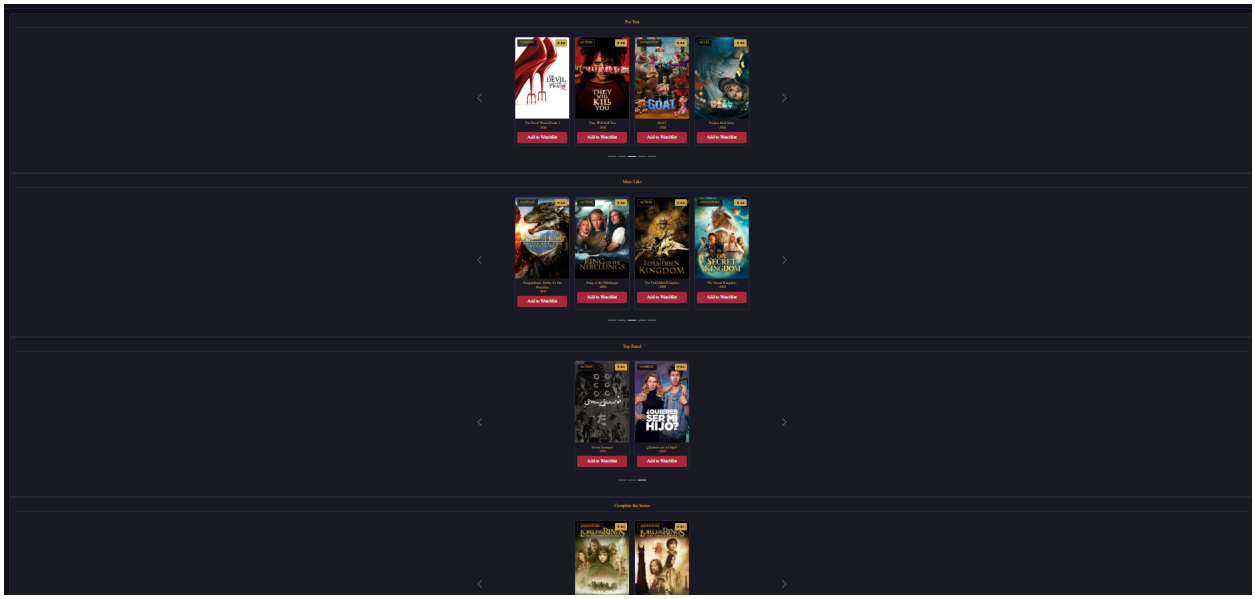


- The purpose of this page is to display the logged-in user's saved movies and allow them to manage their watch progress. It can only be accessed by logged-in users. The key UI elements are a grid of movie cards each with a "Watched" toggle and a "Remove" button. Users can mark movies as watched or remove them entirely from their list. After toggling watched, the card dims to show completion. After removing, the card disappears from the grid instantly.

RECOMMENDATIONS

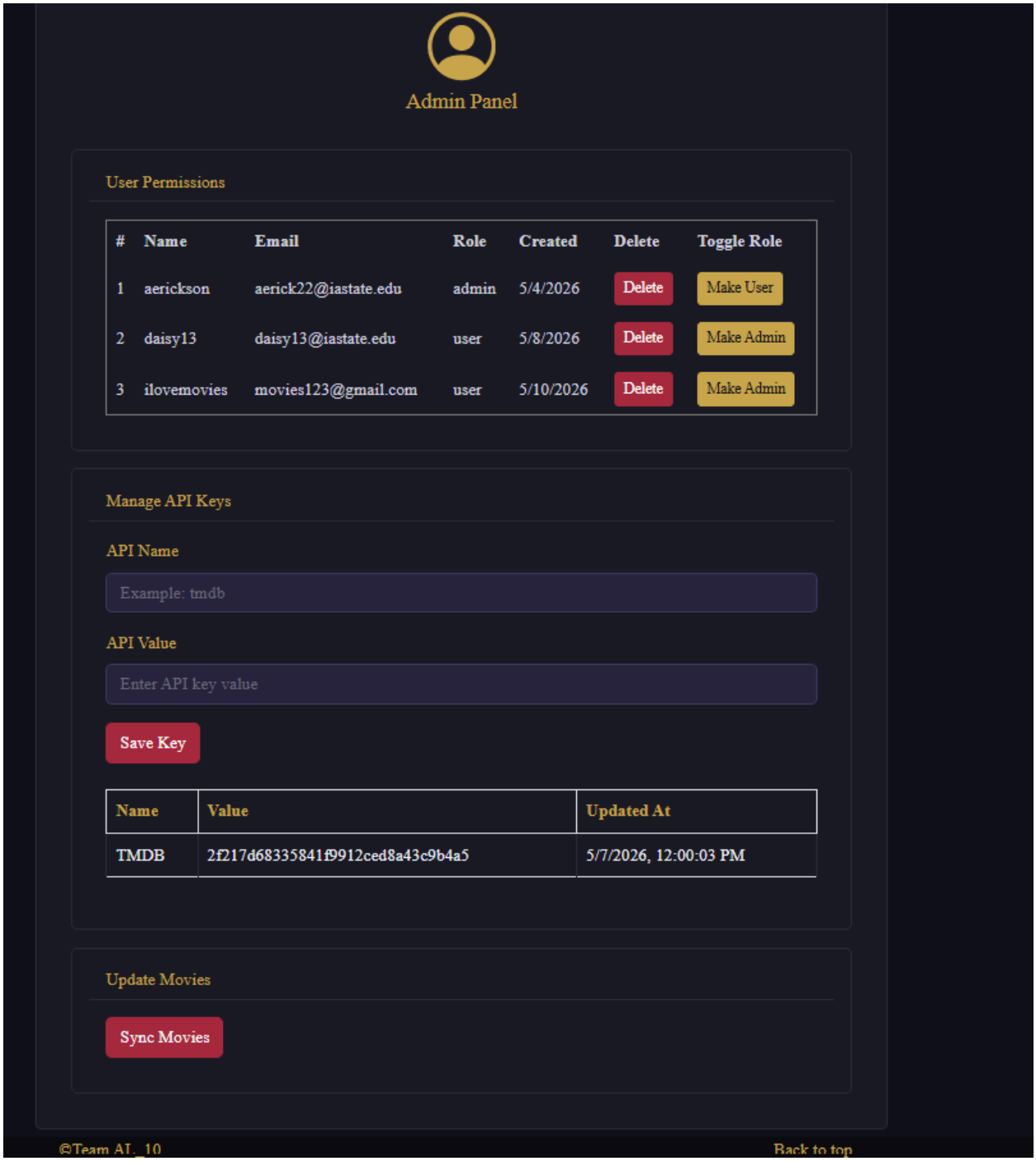
- **The purpose of this page is to surface personalized movie suggestions across four categories based on the user's taste and watchlist. It can only be accessed by logged-in users. The key UI elements are four carousels — For You, More Like, Top Rated, and Complete the Series — each with an "Add to Watchlist" button per card. Users can browse recommendations and add any movie to their watchlist in one click. After adding, the movie is saved to the database if it doesn't exist yet,**

then appended to the user's watchlist, and a success or warning alert is shown.



ADMIN CONTROLS

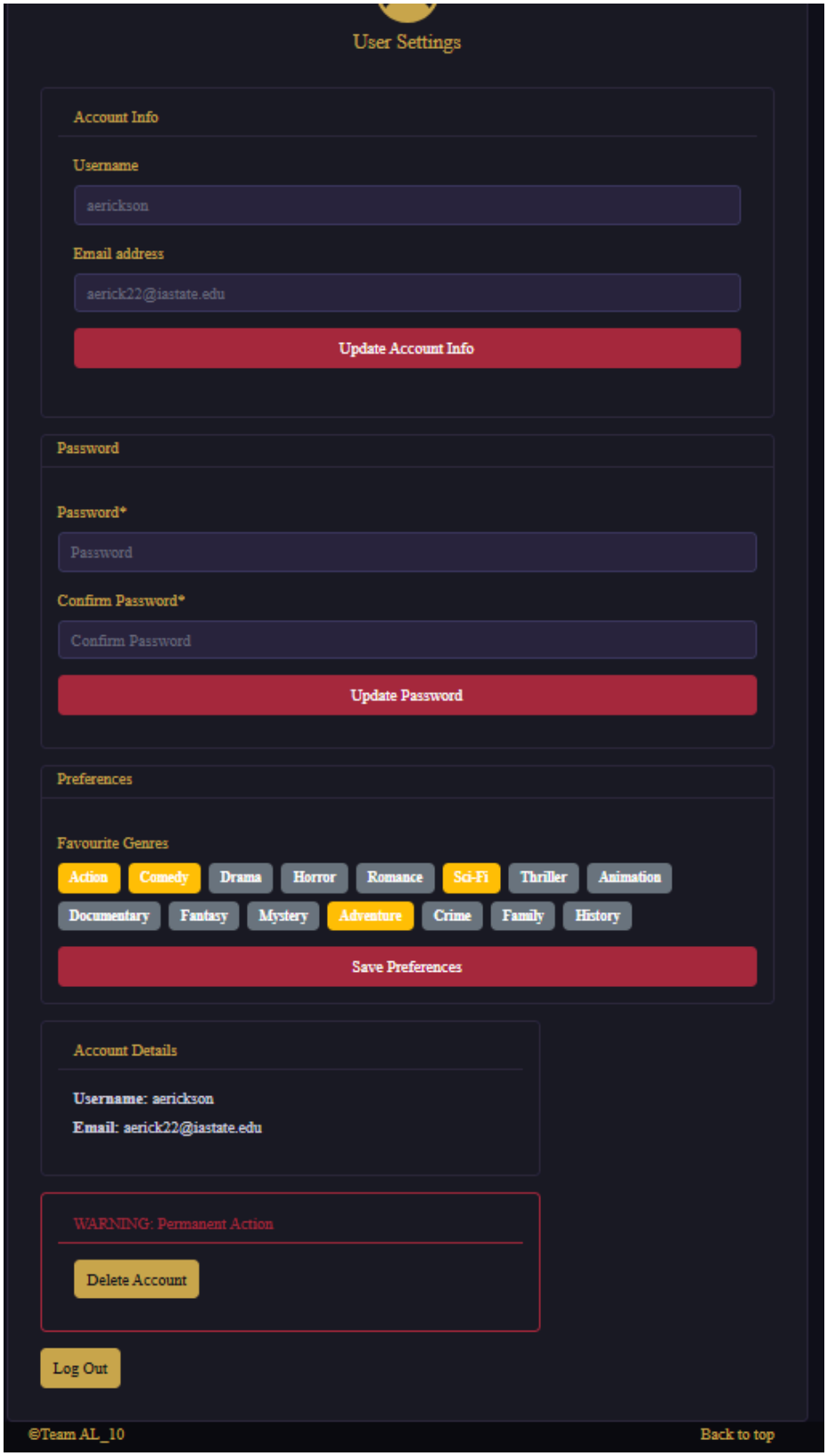
- **The purpose of this page is to give admins full control over users, API keys, and movie data syncing. It can only be accessed by users with the admin role. The key UI elements are a user permissions table with delete and role toggle buttons, an API key form with a keys table, and a Sync Movies button. Admins can delete users, promote or demote them between user and admin roles, create or update API keys, and trigger a full movie sync. After each action a confirmation dialog is shown first, and on success the relevant table refreshes or an alert confirms the outcome.**

- 

SETTINGS

- **The purpose of this page is to allow logged-in users to update their account information, password, genre preferences, and manage their account lifecycle. It can only be accessed by logged-in users. The key UI elements are an account info form, a password form, a genre badge selector, a delete account button, and a log**

out button. Users can update their username and email, change their password, pick their favorite genres, or permanently delete their account. After submitting any form the changes are sent to the server and the user context is updated immediately, and after deleting or logging out the user is cleared and redirected to the home page.


User Settings
Account Info
Username
aerickson
Email address
aerick22@iastate.edu
Update Account Info
Password
Password*
Password
Confirm Password*
Confirm Password
Update Password
Preferences
Favourite Genres
Action
Comedy
Drama
Horror
Romance
Sci-Fi
Thriller
Animation
Documentary
Fantasy
Mystery
Adventure
Crime
Family
History
Save Preferences
Account Details
Username: aerickson
Email: aerick22@iastate.edu
WARNING: Permanent Action
Delete Account
Log Out
©Team AL_10
Back to top

6 Installation and Setup Instructions

This section ensures your project can be successfully run by another developer.

6.1 Frontend Setup

Provide detailed instructions:

- Required Node version: **11.9.0**
- Commands to install dependencies (npm install): **cd frontend -> npm install bootstrap react-bootstrap react-bootstrap-icons react-hook-form react-router-dom**
- Any environment variables needed for the frontend: **Starter Data in data folder**
- How to start the development server (npm run dev) **npm run dev**
- Any special notes (base URLs, API ports)

6.2 Backend Setup

Give clear setup steps:

- Required Node version and global dependencies (nodemon, dotenv) **Node 11.9.0 and Nodemon: 3.1.14**
- Commands to install backend dependencies: **cd backend -> npm install bcrypt body-parser cors express mongodb nodemon uuid**
- Explanation of backend directory structure: **Server.js is the main entry point sets up all the routers from '/api' onwards. db.js just inits the connection to mongodb. In api directory, we have our main routes: Keys.js, Movies.js, Recommendations.js, Review.js, Users.js, Watchlist.js which have all our endpoints to their respective collection in mongo. tmdb_script was used initially to seed our data.**
- Instructions to start the backend server (nodemon server.js or node server.js) **nodemon server.js**
- Any seeding scripts or initialization steps

6.3 Database Setup

Explain how to set up your database:

- Whether using MongoDB Atlas or local MongoDB or MySQL: **MongoDB**
- How to create the required database and collections/tables: Create db called watch_vault, create Keys, Movies, Review, Users, and Watchlists collections and import the json files found in data directory
- Steps to import any seed data:
- Required connection strings and how they fit into the .env file
- Any permissions or network access requirements

6.4 Environment Variables

List all variables needed across frontend and backend, such as:

- PORT for backend server
- FRONTEND URL and CORS domains
- Any API keys used by the app: TMDB key -> `2f217d68335841f9912ced8a43c9b4a5`

7 Contribution Overview

Each member must describe their exact contributions. Avoid vague phrases like “helped with backend”; be explicit and technical.

7.1 Member 1 Contributions

Describe in detail:

- All frontend components built

  Individual: **Search, Review, CreateReviews, Movie Request**

  Contributed: **Home**, **Layout**, **Recommendations/Components** (bug fixes),
- Any styling or UX improvements (Tailwind, animations)

  Contributed to carousel and Layout Header/Footer improvements, any animation in the Search/Review/CreateReviews/MovieRequest pages.
- Backend logic or middleware development

  Tmdb_script, Reviews.js, Movies.js, server.js, and updated vite.config.js

- Database layers created or extended

  Database tables created were Movies and Reviews
- Testing and debugging tasks completed

  General testing conducted through postman, general debugging for files I was responsible for were done. Otherwise bugs were reported from both parties code (Alex and I) and were promptly fixed.
- Any deployment or environment setup contributions

  Created an account for TMDB and used the api under my name for the project. Considered project limitations in README and thus ensured safe API use in setup such that we don't get banned from its limited plan.
- Documentation sections authored

  Sections 1, 3(both), 4, 5(both), 6

7.2 Member 2 Contributions

Describe in detail:

- **All frontend components built:** Watchlist.jsx, Login.jsx, Signup.jsx, Admin.jsx, AdminByPassLogin.jsx, RecommendationsComponents.jsx, ApiKeys.jsx, Layout.jsx, Home.jsx, Recommendations.jsx, UpdateMovies.jsx, and UserPermissions.jsx
- **Styling or UX improvements (Tailwind, animations):** Updated the carousel component from the midterm project
- **Backend logic or middleware development:** Keys.js, Recommendations.js, Users.js, Watchlist.js, db.js, and Server.js
- **Database layers created or extended:** Keys, Users, and Watchlists
- **Testing and debugging tasks completed:** Tested and debugged the Recommendations, Watchlist, and Users modules
- **Deployment or environment setup contributions:** Initialized and configured the project structure, managed dependencies via package.json, set up the Express server and database connection, configured CORS and global middleware, and wrote local setup instructions in the README

- **Documentation sections authored:** Documentation for all frontend and backend files listed above. **Sections 2, 3(both), 5(both)**
- Any deployment or environment setup contributions:
- Documentation sections authored: Both frontend and backend files I made.

8 Challenges Faced

Each member must document the technical challenges they personally faced.

8.1 Member 1 Challenges

For each challenge:

- Describe the root cause or problem encountered
  I had an issue with retrieving specific movies through admin requests. In general the response was slow and would sometimes crash on my end.
- Explain how you diagnosed it (logs, breakpoints, prints, etc.)
  Since I had try and catch statements I was able to realize what error it was spitting out through that, but what was causing it was difficult to diagnose. Eventually I realized it was api specific because I was making 40+requests at a rate which was too fast for the limited plan.
- Provide the solution you implemented
  I limited my request to 10 possible movies at a time and limited the amount of searches that could be done to every 5 seconds so as to not let admins accidentally strain my api instance.
- Describe what you learned from solving the issue
  READ THE FINE PRINT limited api usage is critical when designing even toy systems you can get banned from using their services altogether.

8.2 Member 2 Challenges

Recommendations – Movies Not Pulling Correctly

- **Root cause:** The movie data was being returned in an incorrect format, and the API call was malformed, causing recommendations to fail to load properly.
- **Diagnosis:** Used console.log() and print statements throughout the code to trace the data flow, and used Postman to inspect the API requests and responses directly.
- **Solution:** Fixed the API call and implemented logic to first check if the recommended movie already existed in the database — if so, it was pulled directly from there; if not, it was fetched externally, added to the movies database, and then displayed from there.
- **Lesson learned:** Validating API requests early with a tool like Postman saves significant debugging time, and centralizing data through the database ensures consistency across the application.

UI/Styling Bugs – Layout and Visual Inconsistencies

- **Root cause:** Several UI issues were present: the watchlist was stacking movies horizontally and overflowing off-screen, the carousel was appearing smushed, and there were mismatched colors across components.
- **Diagnosis:** Issues were identified visually by inspecting the rendered application in the browser.
- **Solution:** Fixed all issues by adjusting values in CSS and Bootstrap, correcting the watchlist layout overflow, resizing the carousel, and aligning colors for a consistent look.
- **Lesson learned:** Visual testing is just as important as functional testing — small misconfigured style values can significantly impact usability and the overall user experience.

9 Final Reflections

Each team member reflects individually.

9.1 Member 1 Reflection

Write about:

- What technologies have you become more confident with

  I have become familiar with react and its general flow when designing frontend systems. I was already familiar with cors and postman for backend stuff but I learned how it reacts (with react) and in general how to write more safely since there was less endpoints to consider compared to 3090.
- Which parts of the project were the most rewarding

  The review pages were the most cool since they actually seem real, from the process of creating a review to seeing the individual movies have their own pages is very cool. Also tmdb's api working as I envisioned in my head was a relief, being able to search for movies (through Movie Request) and seamlessly adding them panned out as I wanted to as well, despite the hiccup.
- What would you redesign if starting over

  Establish relationships between tables early in database design, currently we have no relational model in our database, only implied links through table id's.
- New skills gained that apply to future software development

  Understanding of react and a higher understanding of site layouts for global navigation. I am familiar with MySQL and Postgresql so seeing mongodb work in its own way is also refreshing and insightful. In general my understanding of javascript is higher and has been demystified a little despite being very high level magic.

9.2 Member 2 Reflection

Write about:

- What technologies have you become more confident with? **I have become more confident in front-end development throughout the semester. At my job, my boss was very focused on the backend. So I was really good at backend development, but front-end was a challenge.**
- Which parts of the project were the most rewarding:**The recommendations page. It was a struggle getting it to work. I had to do a lot of Postman testing for the TMDB endpoints to get exactly what I was looking for. With that, many errors**

and challenges arise, but when it finally renders correctly and flows based on what's in the watchlist, I feel rewarded.

- What would you redesign if starting over? **I think the watchlist could have been cooler. I think, as it is, it's pretty plain. I was thinking of separating the screen from the watch to be a watch, and when you click the watch button, it appears on the other side. Gives the user some**
- New skills were gained that apply to future software development. **I gained one of the most appropriate stacks in the industry: the MERN stack. A lot of the jobs I see are hiring MERN or something close, so this course will give me that experience. In my current job, I do more clean front-end-to-back-end development and have more knowledge in it.**